

PL-SQL Lab 1

Name : Y Madhav Roll No AM.SC.U4AIE23163

2. Consider the following schema.

Emp(eno, ename, sal)

EMP_PROJ(eno, pno, prjt_hrs).

```
CREATE TABLE Emp (  
    eno INT PRIMARY KEY,  
    ename VARCHAR(50),  
    sal NUMERIC(10,2)  
);  
  
CREATE TABLE EMP_PROJ (  
    eno INT,  
    pno INT,  
    prjt_hrs INT,  
    FOREIGN KEY (eno) REFERENCES Emp(eno)  
);
```

	eno [PK] integer	ename character varying (50)	sal numeric (10,2)
1	1	Prakash	33333.33
2	2	Madhav	45000.00
3	3	Navin	33333.33

	eno integer	pno integer	prjt_hrs integer
1	1	101	5
2	1	102	3
3	2	101	10
4	3	103	2

Write a function ProjectLoad that returns the total project working hours for the given eno.

```
CREATE OR REPLACE FUNCTION ProjectLoad(p_eno INT)
RETURNS INT AS $$
DECLARE
    total_hrs INT;
BEGIN
    SELECT COALESCE(SUM(prjt_hrs), 0)
    INTO total_hrs
    FROM EMP_PROJ
    WHERE eno = p_eno;

    RETURN total_hrs;
END;
$$ LANGUAGE plpgsql;
```

Write a PL/SQL code to update the salary of an employee if the employee earn less than the average salary.

```
DO $$
DECLARE
    avg_sal NUMERIC;
BEGIN
    SELECT AVG(sal) INTO avg_sal FROM Emp;
    UPDATE Emp
    SET sal = avg_sal
    WHERE sal < avg_sal;
END;
$$;
```

Consider the following tables

Item(ino, iname, unit_price)

Transaction(tr_no, ino, qty)

```
CREATE TABLE Item (  
    ino INT PRIMARY KEY,  
    iname VARCHAR(100),  
    unit_price NUMERIC(10,2)  
);  
CREATE TABLE Transaction (  
    tr_no INT PRIMARY KEY,  
    ino INT,  
    qty INT,  
    FOREIGN KEY (ino) REFERENCES Item(ino)  
);
```

ino [PK] integer	iname character varying (100)	unit_price numeric (10,2)
101	Laptop	55000.00
102	Mouse	500.00
103	Keyboard	1200.00
104	Monitor	8500.00
105	USB Pendrive 32GB	600.00
106	Router	2400.00
107	Printer	7000.00
108	Webcam	1800.00

	tr_no [PK] integer	ino integer	qty integer
1	1	101	2
2	2	102	5
3	3	103	3
4	4	101	1
5	5	104	4
6	6	105	2
7	7	106	1
8	8	101	3

Write a function to accept an item no from the user. If transaction has been made for less than 2 times for that item(check from transaction table), delete the item from the Item table.

```
CREATE OR REPLACE FUNCTION DeleteIfLowTxn(p_ino INT)
RETURNS TEXT AS $$
DECLARE
    txn_count INT;
BEGIN
    SELECT COUNT(*) INTO txn_count
    FROM Transaction
    WHERE ino = p_ino;
    IF txn_count < 2 THEN
        DELETE FROM Item WHERE ino = p_ino;
        RETURN 'Item deleted because transactions < 2';
    ELSE
        RETURN 'Item NOT deleted. Enough transactions exist.';
    END IF;
END;
$$ LANGUAGE plpgsql;
```

Consider a Bank database which includes the following tables.

ACCOUNTS(ac_no,br_no, cust_no,ac_type,bal)

BRANCHES(br_no, br_name, loc)

CUSTOMER(cno, cname, c_type)

```

CREATE TABLE ACCOUNTS (
    ac_no INT PRIMARY KEY,
    br_no INT,
    cust_no INT,
    ac_type VARCHAR(20),
    bal NUMERIC(12,2),
    FOREIGN KEY (br_no) REFERENCES BRANCHES(br_no),
    FOREIGN KEY (cust_no) REFERENCES CUSTOMER(cno)
);
CREATE TABLE BRANCHES (
    br_no INT PRIMARY KEY,
    br_name VARCHAR(100),
    loc VARCHAR(100)
);
CREATE TABLE CUSTOMER (
    cno INT PRIMARY KEY,
    cname VARCHAR(100),
    c_type VARCHAR(10)
);

```

	ac_no [PK] integer 	br_no integer 	cust_no integer 	ac_type character varying (20) 	bal numeric (12,2) 
1	1001	101	1	Savings	25000.00
2	1002	101	2	Current	52000.00
3	1003	102	3	Savings	18000.00
4	1004	103	4	Savings	75000.00
5	1005	104	5	Current	12000.00
6	1006	105	6	Savings	90000.00
7	1007	105	7	Savings	15000.00
8	1008	106	8	Current	5000.00
9	1009	107	9	Savings	65000.00

	br_no [PK] integer	br_name character varying (100)	loc character varying (100)
1	101	Hyderabad-Main	Hyderabad
2	102	Hyderabad-Begumpet	Hyderabad
3	103	Vijayawada-Center	Vijayawada
4	104	Vijayawada-South	Vijayawada
5	105	Guntur-Main	Guntur
6	106	Guntur-North	Guntur
7	107	Vizag-Main	Visakhapatnam
8	108	Vizag-Gajuwaka	Visakhapatnam
9	109	Warangal-Main	Warangal

	cno [PK] integer	cname character varying (100)	c_type character varying (10)
1	1	Rohit Kumar	B
2	2	Akhil Reddy	A
3	3	Madhav	B
4	4	Navin Chowdary	A
5	5	Priya Sharma	B
6	6	Kavya Nair	A
7	7	Sanjay Rao	B
8	8	Amrita Varma	B
9	9	Harsha	A

Write a function that accepts a threshold value and a customer number . The program updates the c_type based on the threshold value. Ie. If balance > threshold then class A, else class B.

```

CREATE OR REPLACE FUNCTION UpdateCustomerType(p_threshold NUMERIC, p_cust_no INT)
RETURNS TEXT AS $$
DECLARE
    total_bal NUMERIC;
BEGIN
    SELECT COALESCE(SUM(bal),0)
    INTO total_bal
    FROM ACCOUNTS
    WHERE cust_no = p_cust_no;
    IF total_bal > p_threshold THEN
        UPDATE CUSTOMER
        SET c_type = 'A'
        WHERE cno = p_cust_no;
        RETURN 'Updated to Class A';
    ELSE
        UPDATE CUSTOMER
        SET c_type = 'B'
        WHERE cno = p_cust_no;
        RETURN 'Updated to Class B';
    END IF;
END;
$$ LANGUAGE plpgsql;

```

Write a function called CloseBranch that takes two arguments (the branch to be closed and the branch to take over the accounts) and transfers all accounts at the closing branch to the new branch and removes the closing branch.

```

CREATE OR REPLACE FUNCTION SafeWithdraw(p_acno INT, p_amt NUMERIC)
RETURNS TEXT AS $$
DECLARE
    cur_bal NUMERIC;
BEGIN
    SELECT bal INTO cur_bal
    FROM ACCOUNTS
    WHERE ac_no = p_acno;
    IF cur_bal IS NULL THEN
        RETURN 'Account does not exist';
    END IF;
    IF cur_bal < p_amt THEN
        RETURN 'Insufficient balance. Withdrawal denied.';
    END IF;
    UPDATE ACCOUNTS
    SET bal = bal - p_amt
    WHERE ac_no = p_acno;
    RETURN 'Withdrawal successful.';
END;
$$ LANGUAGE plpgsql;

```

1)Class Questions :

a)Sum of two numbers

```
CREATE OR REPLACE FUNCTION sum_ab(a INT, b INT)
RETURNS INT AS
$$
DECLARE
|   c INT;
BEGIN
|   c := a + b;
|   RETURN c;
END;
$$
LANGUAGE plpgsql;
```

```
--write a function which accepts an int value and return if the value is odd/even
CREATE OR REPLACE FUNCTION odd_even(n INT)
RETURNS TEXT AS
$$
BEGIN
|   IF n % 2 = 0 THEN
|       RETURN 'Even';
|   ELSE
|       RETURN 'Odd';
|   END IF;
END;
$$
LANGUAGE plpgsql;
```

```
--write a function which accepts an employee number and returns salary of respective employee
CREATE OR REPLACE FUNCTION get_salary(emp_no INT)
RETURNS INT AS
$$
DECLARE
|   emp_sal INT;
BEGIN
|   SELECT sal INTO emp_sal
|   FROM Emp
|   WHERE eno = emp_no;
|   RETURN emp_sal;
END;
$$
LANGUAGE plpgsql;
```

```

--write a function which accepts dept no and returns the no of employess in departme
--Emp(eno, ename, sal, deptno)
CREATE OR REPLACE FUNCTION emp_count(dno INT)
RETURNS INT AS
$$
DECLARE
|   cnt INT;
BEGIN
|   SELECT COUNT(*) INTO cnt
|   FROM Emp
|   WHERE deptno = dno;

|   RETURN cnt;
END;
$$
LANGUAGE plpgsql;

```

```

--write a function which takes an e.no and give that employee 10% hike in salary
--if the Average salary of the corresponding dept is greater then the employees current salary
--Emp(eno, ename, sal, deptno)
CREATE OR REPLACE FUNCTION give_hike_if_needed(emp_no INT)
RETURNS TEXT AS
$$
DECLARE
|   emp_sal    NUMERIC;
|   emp_dept   INT;
|   dept_avg   NUMERIC;
BEGIN
|   SELECT sal, deptno INTO emp_sal, emp_dept
|   FROM Emp
|   WHERE eno = emp_no;
|   IF emp_sal IS NULL THEN
|       RETURN 'Employee not found';
|   END IF;

|   SELECT AVG(sal) INTO dept_avg
|   FROM Emp
|   WHERE deptno = emp_dept;
|   IF dept_avg > emp_sal THEN
|       UPDATE Emp
|       SET sal = sal * 1.10
|       WHERE eno = emp_no;

|       RETURN '10% hike applied to employee ' || emp_no;
|   ELSE
|       RETURN 'No hike. Employee salary is not below department average.';
|   END IF;
END;
$$
LANGUAGE plpgsql;

```